ABES INSTITUTE OF TECHNOLOGY, GHAZIABAD

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Data Structure-UNIT-2 (STACK-Introduction)-Lecture-17

BY: AVINASH KUMAR SHARMA

ASSISTANT PROFESSOR, DEPARTMENT OF CSE, ABESIT, GHAZIABAD

STACKS

- **Objectives**

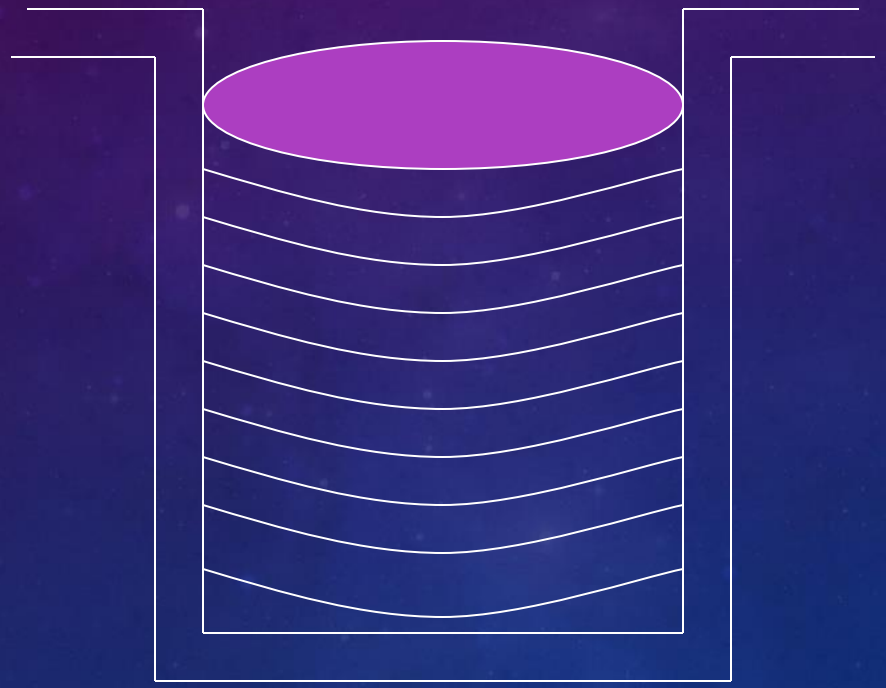
After this lectures (unit-2) you will be able to:

- Describe a stack
- Describe the representation of stack using linear array
- Describe the representation of stack using linear linked list
- Implementation of various operations on stack
- Describe some applications of stacks

STACK

- Stack is one of the commonly used data structures.
- Stack is also called last in first out (LIFO) system
- Stack is a linear list in which insertion and deletion can take place only at one end called top.
- This structure operates in much the same way as stack of trays.

STACK OF TRAYS



- THE FOLLOWING FIGURE ILLUSTRATE A STACK, WHICH CAN ACCOMMODATE MAXIMUM OF 10 ELEMENTS

FIGURE SHOWS STACK AFTER PUSHING ELEMENTS 8,10,12,-5,6

STACK AFTER POPPING TOP TWO ELEMENTS

STACK AFTER PUSHING ELEMENTS 8,10,12,-5,6,9,55

OPERATIONS ON STACKS

- **Createstack(s)—**to create s as an empty stack
- **Push(s, i)--**to push element i onto stack s.
- **Pop(s)—**to access and remove the top element of the stack s
- **Peek(s)—**to access the top element of the stacks without removing it from the stack s.
- **Isfull(s)—**to check whether the stack s is full
- **isempty—**to check whether the stack s is empty

Thanks

ABES INSTITUTE OF TECHNOLOGY, GHAZIABAD

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Data Structure-UNIT-2 (STACK-Introduction)-Lecture-18

BY: AVINASH KUMAR SHARMA

ASSISTANT PROFESSOR, DEPARTMENT OF CSE, ABESIT,GHAZIABAD

REPRESENTATION OF STACK IN MEMORY

- **Representation of stack using array:**

Suppose elements of the stack are integer type and stack can store maximum 10 elements..

```
#define MAX 10
typedef struct
{
   int top;
   int elements[MAX];
}stack;
stack s;
```

- Here we have defined our own data type named stack.
- First element top will be used to index top element
- Array elements hold the elements of the stack
- Last line declares variable s of type stack
- In addition to the previous declaration, we will use the declaration

```
typedef enum {false, true } Boolean;
```

This statement defined new data type named Boolean which can take value false or true.

REPRESENTATION OF STACK IN MEMORY

	0	1	2	3	4	5	6	7	8	9
4 top	8	10	12	-5	6					

	0	1	2	3	4	5	6	7	8	9
2 top	8	10	12							

	0	1	2	3	4	5	6	7	8	9
6 top	8	10	12	-5	6	9	55			

CREATING AN EMPTY STACK

- Before we can use a stack, it is to be initialized.
- As the index of array elements can take any value in the range 0 to MAX-1, the purpose of initializing the stack is served by assigning value -1 to the top of variable.
- This simple task can be accomplished by the following function.

```
Void createstack( stack *ps)
{
ps->top=-1;
}
```

TESTING STACK FOR UNDERFLOW

```
Boolean isempty(stack *ps)
{
   if(ps->top==-1)
        return true;
   else
       return false;
}
or
Boolean is empty(stack *ps)
{
    return ((ps->top==-1)?true:false);
}
```

TESTING STACK FOR OVERFLOW

```
Boolean isfull(stack *ps)
{
   if(ps->top==MAX-1)
        return true;
   else
       return false;
}
or
Boolean is empty(stack *ps)
{
    return ((ps->top==MAX-1)?true:false);
}
```

PUSH OPERATION

Before the push operation, if the stack is empty, then the value of the top will be -1and if the stack is not empty then the value of the top will be the index of the element currently on the top.

Therefore we place the value onto the stack, the value of top is incremented so that it points to the new top of stack, where incoming element is placed.

```
Void push(stack *ps, int value)
{
    ps->top++;
    ps->elements[ps->top]=value;
}
```

POP OPERATION

The element on the top of the stack is assigned to a local variable, which later on will be returned via the return statement.

After assigning the top element to a local variable, the variable top is decremented so that it points to a new top

```
Int pop(stack *ps)
{
    int temp;
    temp=ps->elements[ps->top];
    ps->top--;
  return temp;
}
```

ACCESSING TOP ELEMENT

There may be instances where we want to access the top element of the stack without removing it from the stack.

```
Int peek( stack *ps)
{
  return(ps->elements[ps->top]);
}
```

Thanks

ABES INSTITUTE OF TECHNOLOGY, GHAZIABAD

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Data Structure-UNIT-2 (STACK)-Lecture-19

BY: AVINASH KUMAR SHARMA

ASSISTANT PROFESSOR, DEPARTMENT OF CSE, ABESIT,GHAZIABAD

REPRESENTING A STACK USING A LINKED LIST

A stack represented using a linked list is also known as linked stack.

The array-based representation of stack suffers from following limitations.

- Size of the stack must be known in advance
- We may come across situation when an attempt to push an element causes overflow.

However stack is an abstract data structure can not be full. Hence, abstractly, it is always possible to push an element onto stack. Therefore stack as an array prohibits the growth of the stack beyond the finite number of elements.

DECLARATION OF STACK

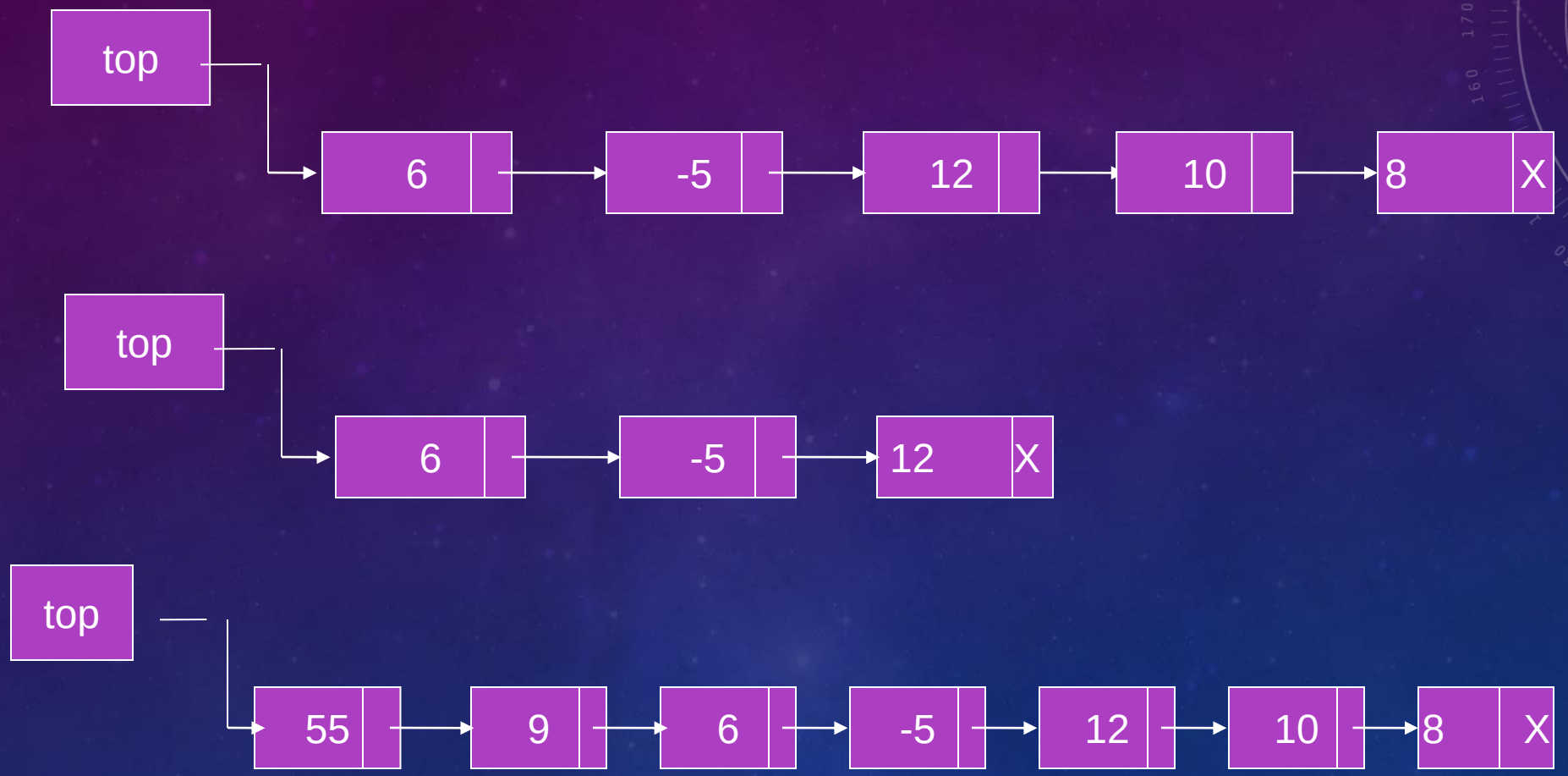
The linked list representation allows a stack to grow to a limit of the computer's memory.

```
Typedef struct nodetype
{
   int info;
   struct nodetype *next;
}stack;
Stack *top;
```

Here I have defined my own data type named **stack**, which is a self-referential structure and whose first element **info** hold the element of the stack and the second element **next** hold the address of the element under it in the stack.

The last line declares a pointer variable **top** of type stack.

REPRESENTATION OF STACK IN MEMORY



CREATING AN EMPTY STACK

Before we can use a stack, it is to be initialized. To initialize a stack, we will create an empty linked list.

The empty linked list is created by setting pointer variable top to value NULL.

```
Void createstack(stack **top)
{
    *top=NULL;
}
```

TESTING STACK FOR UNDERFLOW

```
Boolean isempty(stack *top)
{
   if(top==NULL)
      return true;
   else
      return false;
}
or
Boolean is empty(stack *top)
{
    return ((top==NULL)?true:false);
}
```

TESTING STACK FOR OVERFLOW

Since stack represented using a linked list can grow to a limit of computers memory, there overflow condition never occurs. Hence this operation is not implemented for linked list.

PUSH OPERATION

To push a new element onto the stack, the element is inserted in the beginning of the linked list.

```
void push(stack **top, int value)
{
    stack *ptr;
    ptr=(stack*)malloc(sizeof(stack));
    if(ptr==NULL)
     {
      printf("\n unable to allocate memory for new node…");
      printf("\npress any key to exit..");
      getch();
      return;
     }
    ptr->info=value;
    ptr->next=*top;
    *top=ptr;
}
```

POP OPERATION

To pop an element from the stack, the element is removed from the beginning of the linked list.

```
Int pop(stack **top)
{
    int temp;
    stack *ptr;
    temp=(*top)->info;
    ptr=*top;
    *top=(*top)->next;
    free(ptr);
    return temp;
}
```

ACCESSING TOP ELEMENT

```
Int peek(stack *top)
{
    return(top->info)
}
```

DISPOSE A STACK

Because the stack is implemented using linked lists, therefore it is programmers' job to write the code to release the memory occupied by the stack.

```
Void disposestack(stack **top)
{
    stack *ptr;
    while(*top!=NULL)
      {
      ptr=*top;
      *top=(*top)->next;
      free(ptr);
      }
}
```

Thanks

ABES INSTITUTE OF TECHNOLOGY, GHAZIABAD

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Data Structure-UNIT-2 (STACK)-Lecture-20

BY: AVINASH KUMAR SHARMA

ASSISTANT PROFESSOR, DEPARTMENT OF CSE, ABESIT,GHAZIABAD

APPLICATIONS OF STACKS

Mathematical notation translation

1. Infix to Postfix Conversion
2. Evaluation of Postfix Expression

TYPES OF ARITHMETIC EXPRESSIONS

INFIX, POSTFIX AND PREFIX NOTATIONS

Infix	Postfix	Prefix
A+B	AB+	+AB
A+B-C	AB+C-	-+ABC
(A+B)*(C-D)	AB+CD-*	*+AB-CD

Infix, Postfix and Prefix notations are used in many calculators. The easiest way to implement the Postfix and Prefix operations is to use stack. Infix and prefix notations can be converted to postfix notation using stack.

The reason why postfix notation is preferred is that you don't need any parenthesis and there is no prescience problem.

MATHEMATICAL NOTATION TRANSLATION

Symbol used	**Operation performed**	**Precedence**
^(exponential)	Power	Highest
% (percentage)	Modulus	Highest
*(asterisk), / (slash)	Multiplication, Division	Highest
+ (plus), - (hyphen)	Addition, subtraction	Lowest

INFIX TO REVERSE POLISH NOTATION

Algorithm: Infix_to_PostFix(Q, P)

Suppose **Q** is an arithmetic expression written in infix notation. This algorithm finds the equivalent postfix expression **P**.

1. Push "(" onto STACK, and add ")" to the end of **Q**.

2. Scan Q from left to right and repeat Steps 3 to 6 for each element of Q until the STACK is empty:

3. If an operand is encountered, add it to **P**.

4. If a left parenthesis is encountered, push it onto STACK.

5. If an operator © is encountered, then:

a) Repeatedly pop from STACK and add to **P** each operator (on the top of STACK) which has the same or higher precedence/priority than ©

b) Add © to STACK.

[End of If structure.]

6. If a right parenthesis is encountered, then:

a) Repeatedly pop from STACK and add to **P** each operator (on the top of STACK) until a left parenthesis is encountered.

b) Remove the left parenthesis. [Do not add the left parenthesis to **P**.]

[End of If structure.]

[End of Step 2 loop.]

7. Exit.

CONVERT Q: A+(B * C – (D / E ^ F) * G) * H INTO POSTFIX FORM SHOWING STACK STATUS . NOW ADD “)” AT THE END OF EXPRESSION A+(B * C – (D / E ^ F) * G) * H) AND ALSO PUSH A “(“ ON STACK.

Symbol Scanned	Stack	Expression Y
	(	
A	(	A
+	(+	A
(	(+(	A
B	(+(	AB
*	(+(*	AB
C	(+(*	ABC
-	(+(-	ABC*
(	(+(-(	ABC*
D	(+(-(	ABC*D
/	(+(-(/	ABC*D
E	(+(-(/	ABC*DE
^	(+(-(/^	ABC*DE
F	(+(-(/^	ABC*DEF
)	(+(-	ABC*DEF^/
*	(+(-*	ABC*DEF^/
G	(+(-*	ABC*DEF^/G
)	(+	ABC*DEF^/G*-
*	(+*	ABC*DEF^/G*-
H	(+*	ABC*DEF^/G*-H
)	empty	ABC*DEF^/G*-H*+

EVALUATING MATHEMATICAL EXPRESSIONS

IF **P** IS AN ARITHMETIC EXPRESSION WRITTEN IN POSTFIX NOTATION. THIS ALGORITHM USES STACK TO HOLD OPERANDS, AND EVALUATE **P**.

Algorithm: This algorithm finds the VALUE of **P** written in postfix notation.

1. Add a Dollar Sign "$" at the end of **P**. [This acts as sentinel.]
2. Scan **P** from left to right and repeat Steps 3 and 4 for each element of **P** until the sentinel "$" is encountered.
3. If an operand is encountered, put it on STACK.
4. If an operator © is encountered, then:
   - **a)** Remove the two top elements of STACK, where **A** is the top element and **B** is the next-to—top-element.
   - **b)** Evaluate **B** © **A**.
   - **c)** Place the result of (b) back on STACK.

   [End of If structure.]

   [End of Step 2 loop.]
5. Set VALUE equal to the top element on STACK.
6. Exit.

FOR EXAMPLE:

FOLLOWING IS **POSTFIX** ARITHMETIC EXPRESSION: 5 2 + 3 * 8 4 / - = 5 2 + 3 * 8 4 / - $

Scanned Elements	Stack	Action to do
5	5	Pushed on stack
2	5, 2	Pushed on Stack
+	7	Remove the two top elements and calculate 5 + 2 and push the result on stack
3	7, 3	Pushed on Stack
*	21	Remove the two top elements and calculate 7 * 3 and push the result on stack
8	21, 8	Pushed on Stack
4	21, 8, 4	Pushed on Stack
/	21, 2	Remove the two top elements and calculate 8 / 2 and push the result on stack
-	19	Remove the two top elements and calculate 21 - 2 and push the result on stack
$	**19**	**Sentinel $ encouter ,** Result is on top of the STACK

INFIX TO POSTFIX PROCEDURE

Consider the following infix expression q:

(7-5)*(9/2) → (7-5)*(9/2))

Character scanned	stack	Expression p
	(	
(	((	
7	((	7
-	((-	7
5	((-	7 5
)	(	7 5 -
*	(*	7 5 -
(	(*(	7 5 -
9	(*(	7 5 – 9
/	(*(/	7 5 – 9
2	(*(/	7 5 – 9 2
)	(*	7 5 – 9 2 /
)	Empty	7 5 – 9 2 / *
End of expression		7 5 – 9 2 / *

EVALUATING EXPRESSION IN POSTFIX NOTATION

Consider the following postfix expression p:

7 5 – 9 2 / *

Character Scanned	Stack
7	7
5	7 5
-	2
9	2 9
2	2 9 2
/	2 4.5
*	9
End of expression	9

Thanks